

Striking the Privacy – Utility Balance: Mitigating Execution Time Leakage in Differential Privacy

Presenters: Anh Pham

Advisor: Dr. Johes Bater

Background: The Data Utility – Privacy Dilemma

- **Big data unlocks insights**

- Fuels innovation
- Empowers decision-making processes

- **Privacy concerns**

- Risks of privacy breach

→ need to safeguard the privacy of individuals whose data contributes to these insights

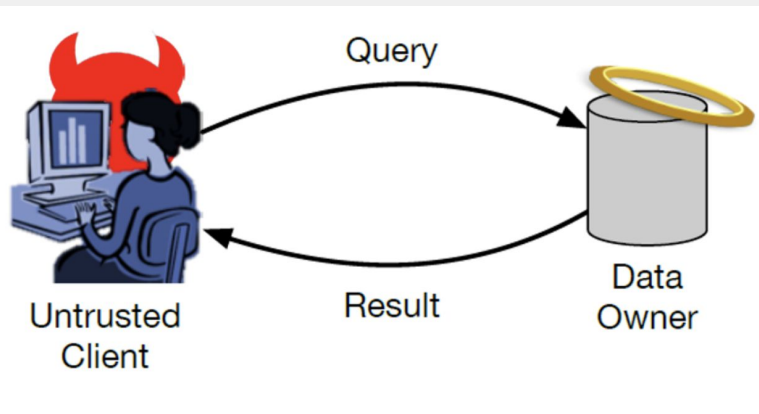
- Privacy measures may impact utility: hardware needs, slower execution, and possible inaccuracies.

→ need to strike a balance data utility – privacy



Definitions and Assumptions

- What counts as a **privacy breach**?
 - Unauthorized access to raw data (out of scope)
 - Using output derived from private data to infer details about an individual→ more subtle, is present in big data analysis
- **Semi-honest** adversaries
 - The players follow protocol instructions but try to exploit private information from private database.



**Infers sensitive information about individuals
from the released output**

**How do we keep
query execution
safe?**

What is a query?

```
SELECT COUNT(*) FROM student WHERE gpa < 3;
```

Scan

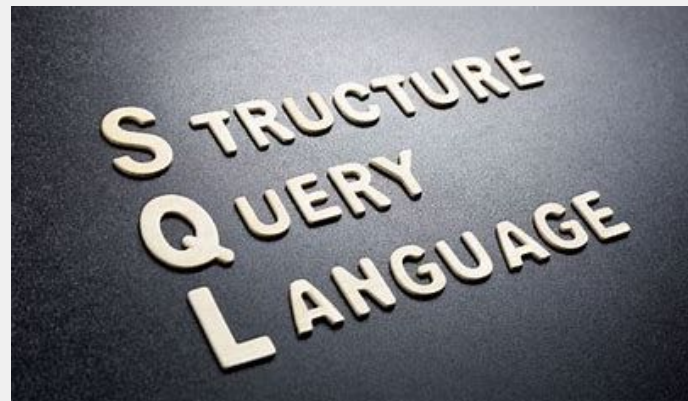
Count

Filter

Order of query execution:

Scan → Filter → Count

- Scan the data in table “student”
- Filter all rows where student’s gpa < 3
- Count no. rows in the filtered data to get the no. students whose gpa < 3
- Return Result = 40
→ 40 students has gpa <3 in this table



Structured Query Language (SQL)

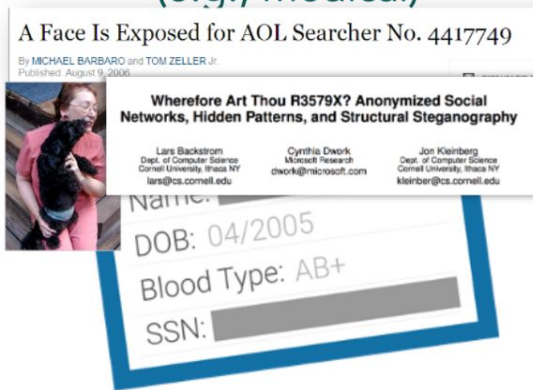
- Common
- Easy to use
- Versatile

Relational Database Management Systems

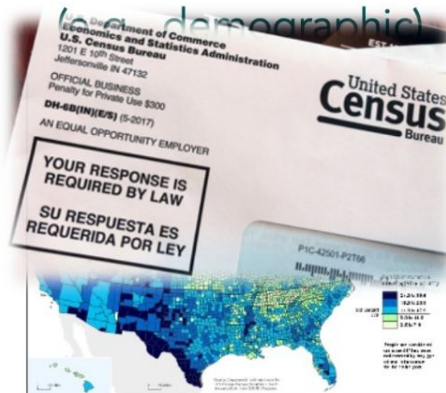
- PostgreSQL
- MySQL
- SQLite
- etc.

Conventional Data Privacy Methods

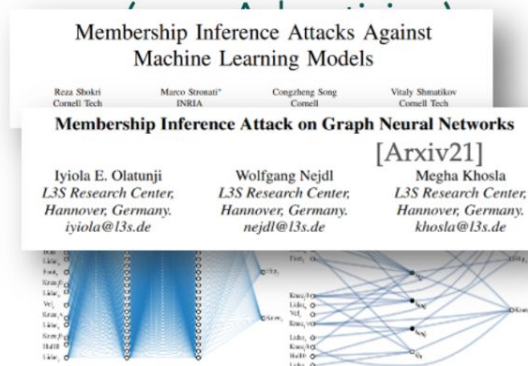
De-identified Records (e.g., medical)



Aggregate Statistics



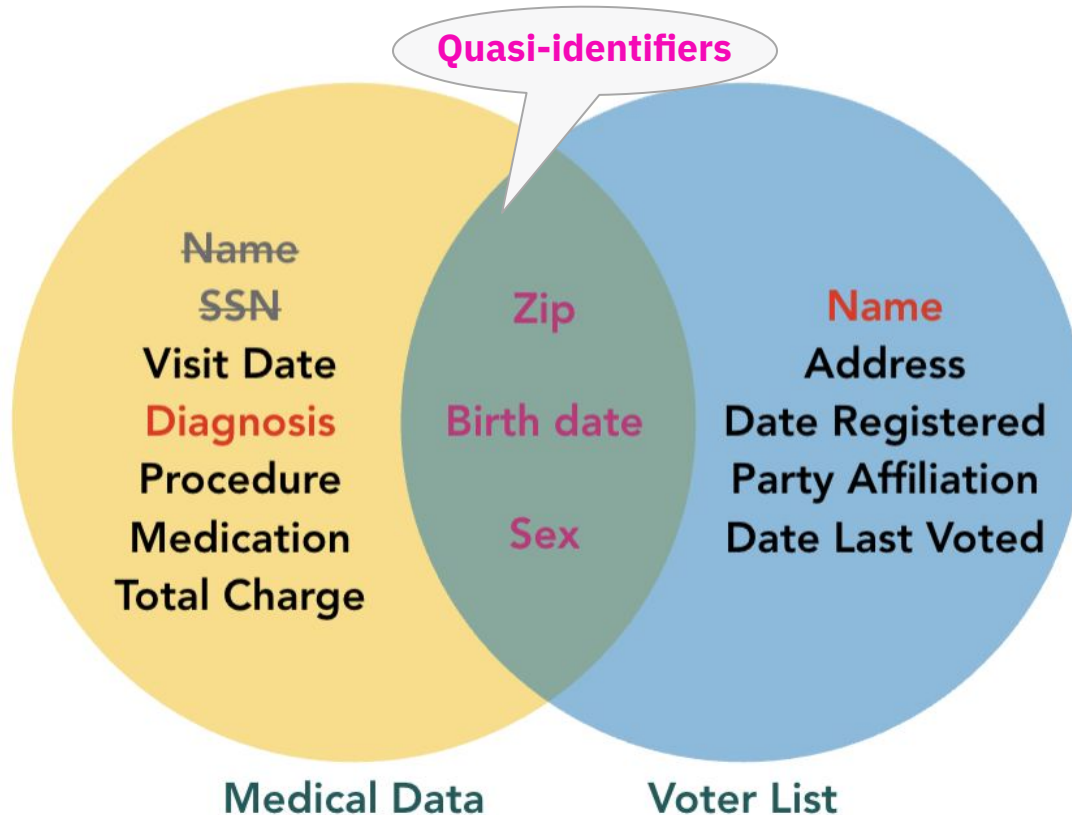
Predictive Models



Fundamental Law of Information Reconstruction

"Overly accurate" estimates of "too many" statistics are not private

The Massachusetts Governor Privacy Breach



Governor of MA **uniquely identified** using Zip Code, Birth Date, and Sex.

Name linked to Diagnosis

[Sweeney IJFUKS 2002]

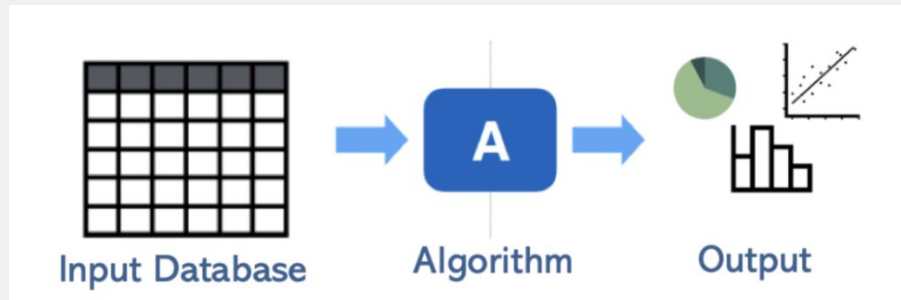
DIFFERENTIAL PRIVACY

- allows insightful analyses
- minimizes privacy risks



Differential Privacy

- **Goal**
 - Protect individual privacy in the database while releasing the output of a valid computation to an untrusted client
 - Adding **controlled noise** to query outputs
→ obscures the influence of any single individual's data on the overall analysis
- **Trade-Offs:** Accuracy vs. Privacy



DP (Differential Privacy) refers to the property of the algorithm e.g. a differentially private algorithm

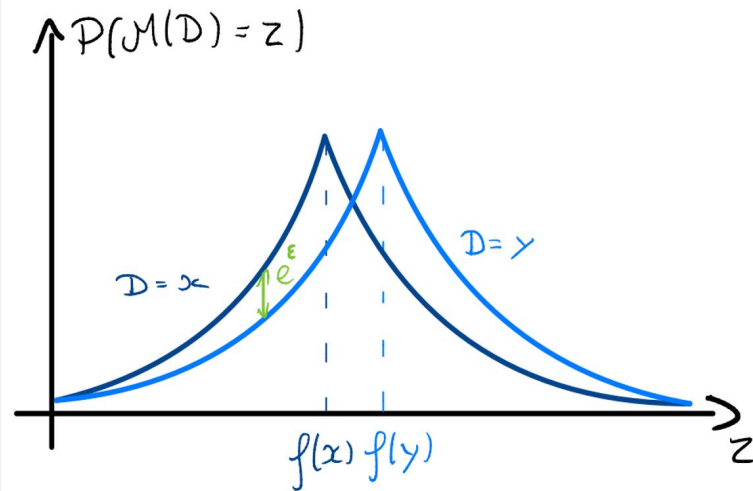
DP Mechanism: some mathematical intuition

- A mechanism $A(D)$ is private if it reveals no more about an individual than a scenario where the individual's data is absent.
- Example:
 - Assume that Alice is a high school student
 - There is mechanism reveals the number of total student absences per semester.
 - If the mechanism gives a similar result whether or not Alice (or any other student) is in D
→ then this mechanism is private.
- Common mechanism for adding noise:
 - Laplace distribution
 - Gaussian (normal distribution)

Laplace Mechanism

- Dataset x contains Alice
- Dataset y does NOT contain Alice
- Dataset y differs from dataset x by only **one row** containing Alice.

Laplace mechanism performed on two adjacent datasets x and y

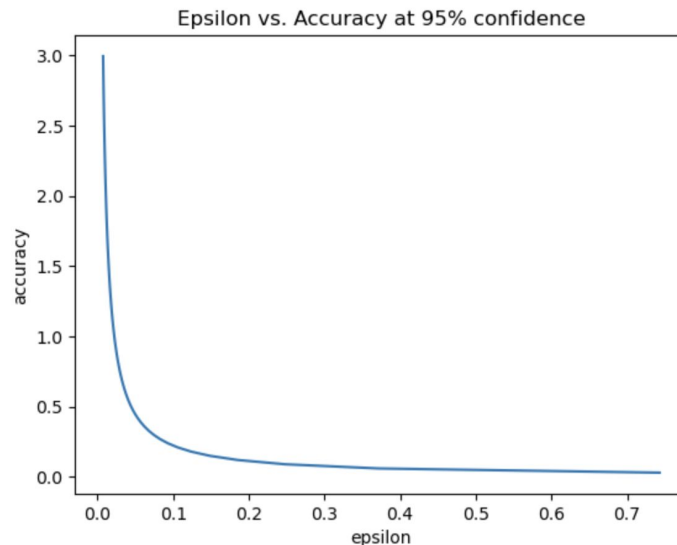


DP Trade-Off: Privacy vs Utility

- Greater privacy corresponds to lower utility.
- Intuitively, the more noise added
→ the more we obscure a single individual's contribution to the data
→ the more private but the less accurate
- Decide noise level via the privacy parameter “epsilon (ϵ)”: the smaller the epsilon, the stronger privacy, the weaker accuracy.

Noisy Answer

$$\tilde{q}(D) = q(D) + \text{Lap}\left(\frac{GS(q)}{\epsilon}\right)$$



An example of adding noise using DP

Scan → Filter → Count → Add Noise

Scan - Integrate data sources with Python



PostgreSQL

Scan CSV files

Scan Postgres database



Python environment
where all privacy efforts
take place

Why Python?

- versatility, usability
- Pandas dataframe

Adding DP noise using openDP

```
SELECT COUNT(*) FROM student WHERE gpa < 3;
```

Real Count = 40

To add noise, first determine **privacy parameters**.

Enter the sensitivity (e.g., 1):

1

Enter the epsilon (e.g., 0.1):

0.5

Select a mechanism for adding noise:

1. Laplace Noise
2. Gaussian Noise
3. Random Noise

Enter your choice (1, 2, or 3): 2

Selected mechanism: Laplace Noise

DP Count with laplace is: 42

Max deviation for count at a 95% confidence level: 6.430

When the laplace distribution scale is 2.0,

the DP estimate differs from the exact estimate

by no more than 6.430 at a 95% confidence level.

```
def privacy_parameters():
    print("In order to add noise using Laplace, Gaussian or Random mechanism,")
    print("Enter the sensitivity (e.g., 1): ")
    sensitivity = float(input())
    print("Enter the epsilon (e.g., 0.1): ")
    epsilon = float(input())
    # Calculate the scale parameter for Laplace and Gaussian distribution
    scale = sensitivity / epsilon
    return sensitivity, epsilon, scale
```

```
# Add Laplace noise for differential privacy
import opendp
from opendp.transformations import *
from opendp.measurements import *

def laplace_noise(result, scale):
    if isinstance(result, int):
        dp_count = make_base_discrete_laplace(scale)
    if isinstance(result, float):
        dp_count = make_base_laplace(scale)
    noisy_result = dp_count(result)
    return noisy_result
```

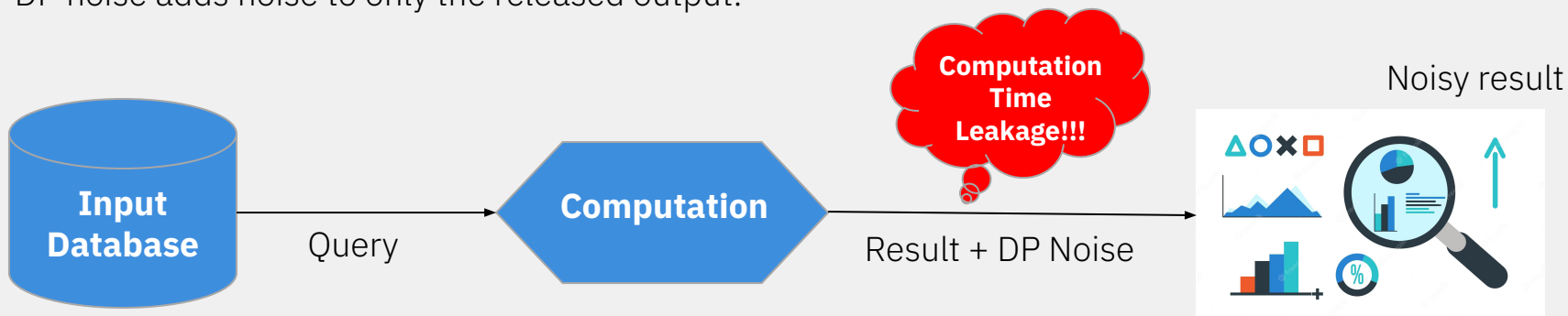
Differential Privacy Limitations

As robust as DP is at protecting privacy while allowing data analysis, DP noise has privacy limitations



DP Limitations: Execution time leakage

- Assumptions: Semi-honest parties follow protocol but try to learn about other inputs as much as they can based on released output or computational time
- Adversaries (such as a data analyst):
 - cannot see raw data in the database
 - can see aggregate statistics
 - can infer information based on output or computational time.
- DP noise adds noise to only the released output.



- Risk of execution time leakage
- Execution time could be used to infer the size and characteristics of the dataset
→ undermines privacy

Addressing execution time leakage in DP

Question: Why don't we just manually add time delay?

`time.sleep()`

Addressing execution time leakage in DP



Question: Why don't we just manually add time delay?

`time.sleep()`

Answer:

Adding time requires knowing exactly how long it takes to process each row.

This can be difficult to do, especially when working with lots of different hardware.

→ Challenge for scalability and consistency

→ Goal: Ensure the time it takes to process queries remains consistent across datasets.

Solution: Data Padding

Principle: In order to be private,
execution should have the same execution time
regardless of the size of the input data.

Objective:
Hide execution time

Scalability:
remains efficient
when n increases

Consistency:
processing time is independent
of size of input

Data Padding - solution for execution time leakage

- What is data padding?

- Add dummy (fake) data to keep the size of the query result the same or close to the original dataset.
- Keep track of fake and real data using dummy bit (boolean 0-1)

```
1. SELECT COUNT(*) FROM student WHERE gpa < 3;
   sid      sname sex  age  year  gpa  dummy
0       4  Sulfate, Barry M.  m   19    2  2.8    1
1       7   Sather, Roberto B.  m   22    4  2.2    1
2       9    Smith, Joyce A.   f   21    4  2.0    1
3      13   Kellerman, S.     f   21    3  2.9    1
4      17  Thorton, James Q.   m   28    4  2.7    1
..      ...                ... ..  ...  ...  ...
99       0                0  0    0    0  0.0    0
100      0                0  0    0    0  0.0    0
101      0                0  0    0    0  0.0    0
102      0                0  0    0    0  0.0    0
103      0                0  0    0    0  0.0    0
```

[104 rows x 7 columns]
Real result is: 40

```
SELECT COUNT(*) FROM student WHERE gpa < 3;
```

Before Pad

Scan → Filter → Count → Add Noise

After Pad

Scan → Filter + Pad → Count → Add Noise

Scan:

Size of table “student” = 104

Filter:

No. students with $\text{gpa} < 3 = 40$

+ Pad

Dummy data = $104 - 40 = 64$

Count & Add Noise as usual

Dummy Data

64

Real Data

40

Full Padding

- **Padded size**

= maximum possible size

- E.g. If the input is size n , then the padded size is also size n .
- Here, input for filter = 104
→ padded size before count has to be 104
→ pad 64 dummy rows

```
1. SELECT COUNT(*) FROM student WHERE gpa < 3;
   sid      sname sex  age  year  gpa  dummy
0      4  Sulfate, Barry M.  m   19    2  2.8    1
1      7  Sather, Roberto B.  m   22    4  2.2    1
2      9    Smith, Joyce A.  f   21    4  2.0    1
3     13  Kellerman, S.      f   21    3  2.9    1
4     17  Thorton, James Q.  m   28    4  2.7    1
...
99      0      0      0      0      0  0.0    0
100     0      0      0      0      0  0.0    0
101     0      0      0      0      0  0.0    0
102     0      0      0      0      0  0.0    0
103     0      0      0      0      0  0.0    0
```

[104 rows x 7 columns]
Real result is: 40

```
SELECT COUNT(*) FROM student WHERE gpa < 3;
```

After Pad

Scan → Filter + Pad → Count → Add Noise

input going
into Filter

=

input going
into Count

Scan:

Size of table "student" = 104

Filter:

No. students with $\text{gpa} < 3$ = 40

+ Pad

Dummy data = $104 - 40 = 64$

Count & Add Noise as usual

Dummy Data

64

Real Data

40

Full Padding

Full padding

- Keep the **execution time constant** across all stages regardless of the output of each stage
- Output of “scan” = 104
Output of “filter” = 40
but data is padded so that data goes in the next stage “count” is still 104
→ execution time remains the same as if the result “filter” is the maximum size 104
→ hide true execution time for the “filter” step
→ curious parties cannot infer information based on execution time

```
SELECT COUNT(*) FROM student WHERE gpa < 3;
```

After Pad

Scan → Filter + Pad → Count → Add Noise

input going
into Filter

=

input going
into Count

Scan:

Size of table “student” = 104

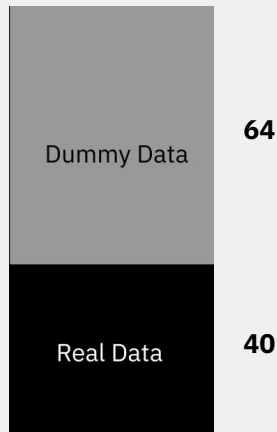
Filter:

No. students with $\text{gpa} < 3 = 40$

+ Pad

Dummy data = $104 - 40 = 64$

Count & Add Noise as usual



Full Padding Trade-offs: Privacy vs. Utility

- Full Padding always increases execution time to the worst case (the largest size n)
- Full Padding: strong privacy, weak utility



Do we always need to pad
to the maximum size?

Full Padding

Do we always need to pad to the maximum size?

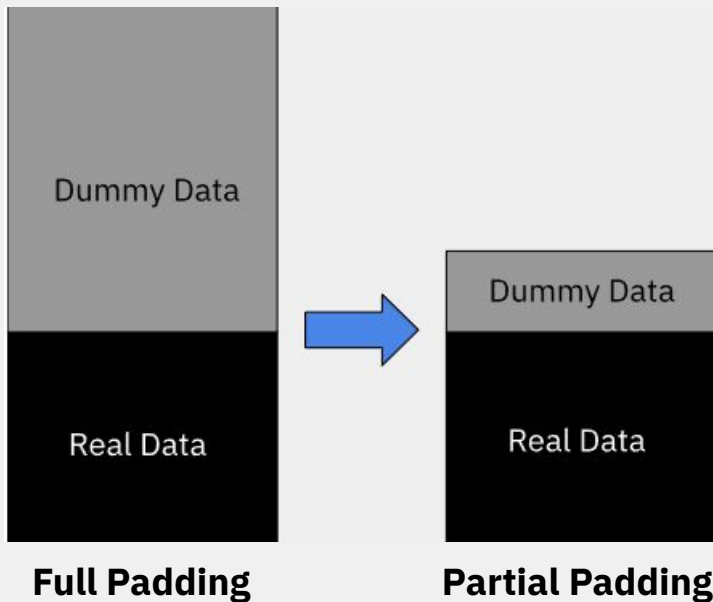
Answer: Not necessarily!

- We can choose our trade-off between privacy–utility
- e.g. sacrifice a bit of privacy for increased utility
- Partial Padding or DP Padding

Partial Padding (DP Padding)

- Depending on how much privacy we need, decide padded size
- e.g. 25% dummy data, 50% dummy data, 75% dummy data

- Could be more balanced method to protect true execution time
- Flexibility to determine the trade-off privacy – utility



```
#Partially pad so that the data contains k*100 % dummy values
def DP_pad_data(filtered_df, original_df, k, track_column):
    if not 0 < k < 1:
        raise ValueError("The value of k should be between 0 and 1.")
    min_size = len(filtered_df)
    max_size = len(original_df)
    if min_size >= max_size:
        raise ValueError("filtered_df should have fewer rows than original_df.")
    size_diff = max_size - min_size

    # Calculate the number of rows to pad, but ensure it does not exceed size_diff
    n_rows = int((min_size * k) / (1 - k))
    max_k = size_diff / (min_size + size_diff)

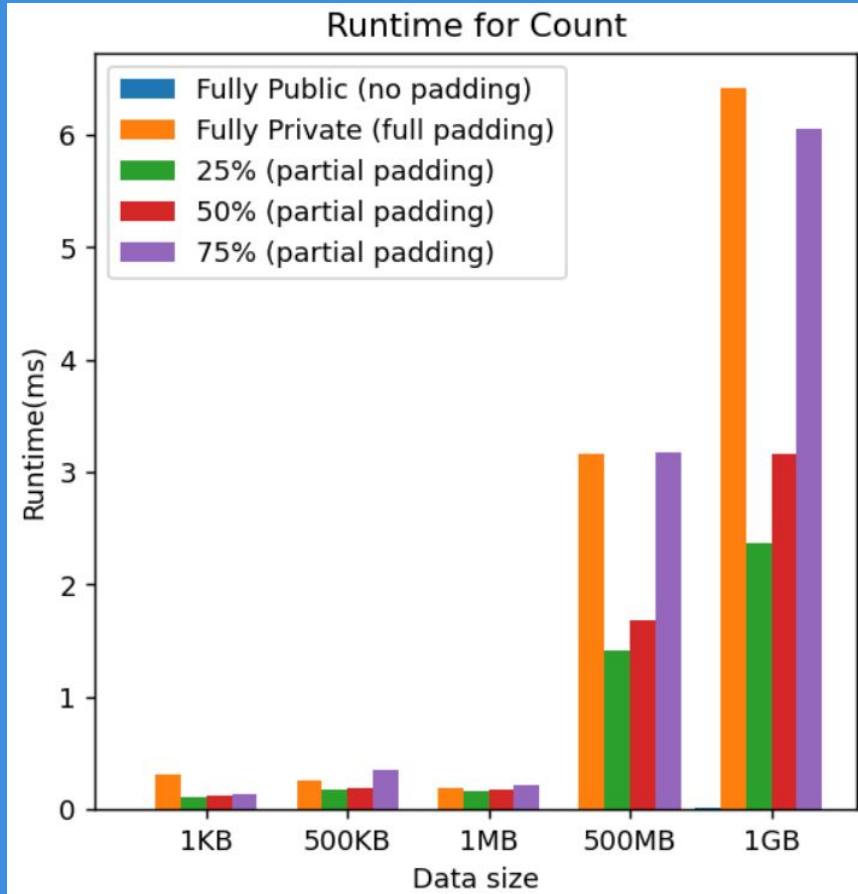
    if n_rows > size_diff:
        print(f"Sorry, padding up to {k*100:.2f}% dummy in this dataset is not applicable. Your data will be padded to the maximum input size of {max_size}."
              f"Maximum effective k: {max_k:.3f}, beyond this value the data will be Runtime is now equal to fully private count.")
        pad_rows = min(n_rows, size_diff)
        size_df_padded = min_size + pad_rows

    # Pad with dummy data
    zeros_df = pd.DataFrame(0, index=range(pad_rows), columns=filtered_df.columns)
    df_padded = pd.concat([filtered_df, zeros_df], ignore_index=True)

    zero_filled = ~(df_padded == 0).all(axis=1)
    df_padded[track_column] = zero_filled.astype(int)

    return df_padded
```


Performance Comparison



- **Performance differs for each level of padding**
 - **Fully private (full padding):**
93 to 1054 times slower than fully public
 - **Partial Padding at 25% dummy data:**
54 to 471 times slower than fully public

All measured in milliseconds (ms = 0.001s)
- **Determine padding based on privacy – utility trade-off**



Future improvements

- **Optimized Padding:** An algorithm that optimizes the amount of data to pad based on DP principles (automatic k).
- **Dynamic Epsilon:** Auto-select optimal parameters, such as noise intensity, for different query types (now it's user-defined).
- **Parallel Processing:** Process multiple tasks at the same time, ensure consistent execution time.



Acknowledgements

- Dr. Johes Bater
- Tufts Security & Privacy Lab
- VERSE – Tufts REU Program
- DIAMONDS – National Science Foundation (grant #2149871)
- Papers
 - *Bater, Johes & Elliot, Gregory & Eggen, Craig & Goel, Satyender & Kho, Abel & Rogers, Jennie. (2016). SMCQL: Secure Query Processing for Private Data Networks.*
 - *Bater, Johes & He, Xi & Ehrich, William & Machanavajjhala, Ashwin & Rogers, Jennie. (2018). Shrinkwrap: efficient SQL query processing in differentially private data federations. Proceedings of the VLDB Endowment. 12. 307-320. 10.14778/3291264.3291274.*



Summary: Mitigating Execution Time Leakage in Differential Privacy

- Motivation
 - Importance of data sharing
 - Privacy vs. Utility trade-off
- Background
 - Conventional data privacy shortcomings
 - Privacy assumptions
- Differential Privacy offers better privacy
 - protects individual privacy
 - adds controlled noise
 - trade-off privacy–utility
- Differential Privacy limitation:
 - execution time leakage
- Proposed solutions for execution time leakage:
 - Manual time delay: not effective
 - Data Padding and Trade-offs
 - Full Padding:
 - strong privacy, hide execution time
 - weak utility (e.g. slow runtime)
 - Partial Padding (DP Padding):
 - more balanced trade-off
 - allows flexibility
- Visualization: Performance comparison for different padding levels
- Future research/improvements



Questions